

Checklist

Rearchitecting Platforms

1. Confirm a Re-architecture Is Actually Needed

- ☐ Identify architectural limitations that incremental improvements can no longer solve.
- ☐ Confirm the current architecture is limiting one or more of these areas:
 - ☐ Feature delivery
 - ☐ Reliability
 - ☐ Security
 - ☐ Efficiency
- ☐ Determine whether business growth or increased platform adoption justifies architectural change.
- ☐ Prefer incremental rearchitecture over building a completely separate “v2” unless there is a compelling reason not to.
- ☐ Avoid combining a new-product problem with a major architecture redesign if possible.
- ☐ Identify the maturity level of the platform:
 - ☐ Scrappy
 - ☐ Scalable
 - ☐ Robust
- ☐ Ensure the engineering mindset matches the platform’s current needs:
 - ☐ Pioneer — experimentation and rapid product discovery
 - ☐ Settler — scaling and operationalizing successful capabilities
 - ☐ Town planner — reliability, security, efficiency, and long-term robustness

2. Define the Final Architecture Goals

- ☐ Use roughly a **3–5-year planning horizon** for the target architecture.
- ☐ Think ambitiously about the desired end state before filtering for feasibility.
- ☐ Define substantial improvements in all four system-capability areas.

Features

- ☐ Identify valuable features the current architecture struggles to support.
- ☐ Determine which new capabilities the rearchitecture should unlock.

Efficiency

- ☐ Estimate whether the new architecture can materially reduce operating costs.
- ☐ Identify opportunities to improve performance as workloads grow.
- ☐ Look for performance improvements that could enable new product capabilities.

Reliability

- ☐ Identify recurring operational problems in the current platform.
- ☐ Define how the new architecture will reduce failures and operational burden at greater scale.

Security

- ☐ Identify significant security or compliance weaknesses.
- ☐ Determine whether the new architecture can reduce the risk of a breach.
- ☐ Determine whether compliance can become easier or less expensive.

3. Simplify the Overall Architecture

- ☐ Identify adjacent platforms or customer systems with overlapping capabilities.
- ☐ Determine whether the new platform can absorb or replace any of them.
- ☐ Look for shadow platforms or “similar but different” systems that could eventually be consolidated.
- ☐ Identify in-house components that duplicate mature open-source or vendor solutions.
- ☐ Evaluate whether replacing an existing component would unlock significantly greater capabilities.

4. Evaluate Major OSS or Vendor Bets

Before architecting around a major external technology:

- ☐ Confirm there is a strong adjacent business need that justifies the investment.
- ☐ Verify that the current platform has meaningful capability gaps the new technology would solve.

- ☐ Evaluate the ecosystem's trajectory and likelihood of continued adoption.
- ☐ Determine whether the technology will still be viable over the expected 5–10-year architecture lifespan.
- ☐ Estimate migration and retraining costs.
- ☐ Avoid switching technologies primarily because something new is fashionable.

5. Build Security Into the Architecture

- ☐ Treat security as a core platform capability rather than a separate activity.
- ☐ Design systems to remain resilient even when attacks or failures occur.
- ☐ Prefer architectural protections that do **not depend on human behavior**.
- ☐ Create a meaningful separation between users and security hazards.
- ☐ Eliminate hazards where possible.
- ☐ Reduce hazards when complete elimination is impractical.
- ☐ Minimize the potential impact of failures or attacks.
- ☐ Minimize manual intervention required to work around hazards.
- ☐ Preserve the ability to change and innovate after security controls are introduced.

Secure-by-Default Platform Capabilities

- ☐ Automated testing
- ☐ Standardized deployment tooling / infrastructure as code
- ☐ Configuration-management patterns
- ☐ Token and secrets management
- ☐ Standardized observability and distributed tracing
- ☐ Standard service and web frameworks
- ☐ Common authentication and authorization middleware
- ☐ Compute platforms with declarative access controls
- ☐ Tenant-isolated architecture where appropriate

Security Opportunities to Evaluate

- ☐ API protection
- ☐ XSRF/CSRF protection
- ☐ Security-header validation
- ☐ Rate limiting, throttling, and tarpitting
- ☐ IP geofencing where appropriate
- ☐ DDoS protection

- ☐ Web application firewall integration
- ☐ Authentication and authorization
- ☐ Certificate management
- ☐ Secure network protocols
- ☐ Static analysis
- ☐ Dynamic analysis
- ☐ Fuzzing
- ☐ Penetration testing
- ☐ Resilience/chaos testing
- ☐ Smoke testing

6. Create Paved Paths

- ☐ Identify recurring security or operational work currently pushed onto application teams.
- ☐ Convert safe practices into supported platform defaults.
- ☐ Make the safest option the easiest option.
- ☐ Allow teams to opt out only when necessary.
- ☐ Reduce application teams' cognitive load rather than adding more checklists and manual processes.
- ☐ Ask which security tasks teams repeatedly perform during software delivery.
- ☐ Identify security mechanisms that should become standard platform requirements.
- ☐ Review security incidents and near misses for opportunities to automate protections.

7. Establish Rearchitecture Guardrails

Compatibility

- ☐ Avoid backward-incompatible API changes whenever possible.
- ☐ If breaking changes are unavoidable, treat them as a major-version migration.
- ☐ Give users sufficient time to migrate before deprecating the old interface.
- ☐ Decide how many platform versions can realistically be supported simultaneously.

Testing

- ☐ Maintain strong unit and integration testing.
- ☐ Test downstream/customer dependencies where practical.
- ☐ Use modern testing techniques, such as property-based testing and fuzzing, where appropriate.
- ☐ Use synthetic monitoring for production validation.
- ☐ Do not rely on “we can roll back quickly” as the primary testing strategy.
- ☐ Use shadow or canary deployments for risky foundational changes.

Lower Environments

- ☐ Maintain a reasonably stable pre-production environment.
- ☐ Complete internal validation before asking customers to test.
- ☐ Use the environment for final integration validation.
- ☐ Avoid repeatedly sending obviously broken releases downstream.

Rollout Strategy

- ☐ Use canary releases where appropriate.
- ☐ Roll changes out gradually across machine/customer tranches.
- ☐ Release to subsets of customers before full deployment.
- ☐ Use beta releases where appropriate.
- ☐ Consider staying slightly behind the newest OSS version when stability and support matter more than novelty.
- ☐ Maintain a deliberate upgrade cadence to avoid falling out of security-support windows.

8. Calculate Migration Costs

- ☐ Create a realistic migration plan alongside the architecture plan.
- ☐ Identify every major group of existing users that must migrate.
- ☐ Estimate the engineering effort required from customer/application teams.
- ☐ Estimate migration duration.
- ☐ Identify compatibility requirements during the transition.
- ☐ Estimate the operational costs of running old and new architectures simultaneously.
- ☐ Include data migration and transformation costs.
- ☐ Include changes to APIs, SDKs, tooling, and integrations.
- ☐ Include migration support and documentation work.
- ☐ Validate migration assumptions with product and engineering managers.

- ☐ Confirm that the expected business value outweighs the migration cost.

9. Identify Major 12-Month Wins

For every major phase of a multi-year rearchitecture:

- ☐ Identify at least one valuable capability deliverable within **12 months**.
- ☐ Ensure the deliverable demonstrates an important property of the new architecture.
- ☐ Find at least one application/customer team willing to use it.
- ☐ Put part of the new architecture into production.
- ☐ Ensure it serves a real customer or production load.

Use Three Levels of Success

- ☐ **Goal 1 — Audacious:** Deliver a major business improvement that meaningfully advances the chosen business area.
- ☐ **Goal 2 — Valuable fallback:** Deliver a smaller but still meaningful improvement that the old architecture could not easily provide.
- ☐ **Goal 3 — Production proof:** Get components of the new architecture running in production with real load.
- ☐ Revisit and reset these goals approximately every year.
- ☐ If no incremental production value can be delivered within 12 months, reassess the architecture and its viability.

10. Manage Pioneer Work Carefully

When rapid experimentation is needed on an otherwise robust platform:

- ☐ Allow a small pioneer team to explore new capabilities quickly.
- ☐ Make it explicit that successful experimental capabilities will eventually integrate into the main platform.
- ☐ Avoid allowing a permanent duplicate “shadow platform” to emerge.
- ☐ Involve existing platform engineers early enough to prepare for integration.
- ☐ Give pioneer teams freedom to move quickly without requiring production-grade completeness immediately.
- ☐ Define an expected integration or transition path before the experiment becomes critical.

- ☐ Recognize and preserve useful contributions from both the pioneer and established platform teams.

11. Build Organizational Support

- ☐ Determine whether the rearchitecture requires additional headcount.
- ☐ Identify internal transfers that may be needed.
- ☐ Identify work required from customer/application teams.
- ☐ Identify migrations or integrations other teams must complete.
- ☐ Estimate whether the work will temporarily slow short-term feature delivery.
- ☐ Assess the reputational risk of failing to meet major commitments.
- ☐ Present both business benefits and costs to leadership.
- ☐ Obtain leadership commitment for a multi-year investment.
- ☐ Make sure leaders understand that the initiative may need protection through reorganizations, layoffs, or changing priorities.
- ☐ Be willing to wait if the business case is not yet strong enough.

12. Staff the Rearchitecture Appropriately

- ☐ Do not automatically assign newly hired engineers to lead the rearchitecture.
- ☐ Use new hires initially to provide feedback and challenge assumptions.
- ☐ Give them time to understand the existing platform, customers, organization, and constraints.
- ☐ Keep experienced engineers who deeply understand the historical context involved.
- ☐ Build strong relationships between new and long-tenured engineers before transferring major architectural ownership.

13. Final Go/No-Go Review

Before committing major resources:

- ☐ Is the current architecture genuinely constraining business value?
- ☐ Are the long-term benefits substantial across features, reliability, security, or efficiency?
- ☐ Is incremental rearchitecture preferable to replacing the entire system?
- ☐ Are migration costs understood?
- ☐ Is there a credible 3-5-year direction?

- ☐ Is there a meaningful 12-month deliverable?
- ☐ Is there at least one committed customer/application partner?
- ☐ Can changes be rolled out without major disruption?
- ☐ Are compatibility, testing, and rollout guardrails defined?
- ☐ Is security improved by design and by default?
- ☐ Is the organization prepared to fund and protect the effort?
- ☐ Is leadership prepared to support the work through changing business conditions?
- ☐ Is the expected benefit clearly greater than the cost and risk?

14. Ongoing Rearchitecture Review

- ☐ Reassess architecture goals annually.
- ☐ Reevaluate customer and business priorities.
- ☐ Review migration progress.
- ☐ Measure actual reliability, efficiency, security, and feature-delivery improvements.
- ☐ Verify that new architecture components are carrying real production load.
- ☐ Remove or deprecate old architecture as customers migrate.
- ☐ Consolidate duplicate platforms whenever possible.
- ☐ Stop, revise, or delay work when evidence no longer supports the original architecture hypothesis.
- ☐ Continue delivering incremental business value throughout the rearchitecture rather than waiting for a “big bang” completion.